

```
In [2]: class Node:
        """Node representing a single student record."""
        def __init__(self, roll_no, name, marks):
            self.roll_no = roll_no
            self.name = name
            self.marks = marks
            self.next = None

        class StudentLinkedList:
            """Singly Linked List to manage student records."""
            def __init__(self):
                self.head = None

            def add_student(self, roll_no, name, marks):
                """Add a new student record at the end."""
                new_node = Node(roll_no, name, marks)
                if self.head is None:
                    self.head = new_node
                else:
                    temp = self.head
                    while temp.next:
                        temp = temp.next
                    temp.next = new_node
                print(f"Student {name} added successfully!")

            def delete_student(self, roll_no):
                """Delete a student record by Roll Number."""
                temp = self.head
                prev = None

                while temp:
                    if temp.roll_no == roll_no:
                        if prev:
                            prev.next = temp.next
                        else:
                            self.head = temp.next
                        print(f"Record with Roll No {roll_no} deleted successfully")
                        return
                    prev = temp
                    temp = temp.next

                print(f"Record with Roll No {roll_no} not found!")

            def update_student(self, roll_no, new_name=None, new_marks=None):
                """Update a student's name or marks."""
                temp = self.head
                while temp:
                    if temp.roll_no == roll_no:
                        if new_name:
                            temp.name = new_name
                        if new_marks is not None:
                            temp.marks = new_marks
                        print(f"Record with Roll No {roll_no} updated successfully")
```

```

        return
        temp = temp.next
    print(f"Record with Roll No {roll_no} not found!")

def search_student(self, roll_no):
    """Search for a student by Roll Number."""
    temp = self.head
    while temp:
        if temp.roll_no == roll_no:
            print(f"Found -> Roll No: {temp.roll_no}, Name: {temp.name}")
            return
        temp = temp.next
    print(f"Record with Roll No {roll_no} not found!")

def sort_records(self, key="roll_no", ascending=True):
    """Sort records based on Roll Number or Marks."""
    if self.head is None or self.head.next is None:
        return

    # Convert linked list to list for sorting
    nodes = []
    temp = self.head
    while temp:
        nodes.append(temp)
        temp = temp.next

    reverse = not ascending
    if key == "marks":
        nodes.sort(key=lambda x: x.marks, reverse=reverse)
    else:
        nodes.sort(key=lambda x: x.roll_no, reverse=reverse)

    # Rebuild linked list
    self.head = nodes[0]
    temp = self.head
    for node in nodes[1:]:
        temp.next = node
        temp = node
    temp.next = None

    print(f"Records sorted by {key} in {'ascending' if ascending else 'descending'}")

def display(self):
    """Display all student records."""
    if self.head is None:
        print("No records to display!")
        return

    print("\nStudent Records:")
    print("Roll No\tName\tMarks")
    print("-" * 25)
    temp = self.head
    while temp:
        print(f"{temp.roll_no}\t{temp.name}\t{temp.marks}")
        temp = temp.next
    print("-" * 25)

```

```
# ----- Example Usage -----  
if __name__ == "__main__":  
    s_list = StudentLinkedList()  
    s_list.add_student(101, "Amit", 85)  
    s_list.add_student(102, "Priya", 92)  
    s_list.add_student(103, "Rohan", 78)  
    s_list.display()  
    s_list.update_student(102, new_marks=95)  
    s_list.search_student(103)  
    s_list.sort_records(key="marks", ascending=False)  
    s_list.display()  
  
    s_list.delete_student(101)  
    s_list.display()
```

Student Amit added successfully!
Student Priya added successfully!
Student Rohan added successfully!

Student Records:

Roll No	Name	Marks
101	Amit	85
102	Priya	92
103	Rohan	78

Record with Roll No 102 updated successfully!
Found -> Roll No: 103, Name: Rohan, Marks: 78
Records sorted by marks in descending order!

Student Records:

Roll No	Name	Marks
102	Priya	95
101	Amit	85
103	Rohan	78

Record with Roll No 101 deleted successfully!

Student Records:

Roll No	Name	Marks
102	Priya	95
103	Rohan	78